

ReturnDesk

Full-stack take-home assignment · 24 hours · Software Developer Intern

The Product

ReturnDesk is a returns desk for a small online store. When a customer wants to return or replace something they bought, they raise a request. A support agent then works through it: reviews it, approves or rejects it, decides whether the customer gets a refund, a replacement or store credit, leaves notes along the way, and finally closes it out.

Your job is to build ReturnDesk. This is a standalone product brief, not a Frido project. Treat it the way you would treat building your own product.

You have 24 hours from the time this document reaches you. The scope is deliberately wider than a comfortable 24-hour build. We are not expecting everything to be finished. We want to see what you choose to build first and how well you build it.

How You Build It Is Your Decision

This brief describes what ReturnDesk must do and the rules it must obey. It does not tell you how to build it.

The database schema, the shape of your API, how you name and organise your endpoints, how you structure responses and report errors, and how you lay out your project are all yours to decide. Those decisions are a large part of what we are assessing. Explain them in your README and be ready to defend them in the technical round.

Technology

Mandatory

- Next.js (App Router) with React for the frontend.
- Node.js on the backend. Next.js route handlers or a separate Express service are both acceptable.
- PostgreSQL. Data must survive a server restart. In-memory stores, JSON files and browser storage will not be accepted.
- Tailwind CSS for styling.

Your choice

- JavaScript or TypeScript.
- Any query layer: raw SQL, Prisma, Drizzle, Knex or similar.
- Any Postgres you like: local, Docker, Supabase, Neon or Render.

Not allowed

- Starting from a ready-made admin, ticketing or CRUD template repository. Starting from create-next-app is expected and fine.
 - Auto-generated backend-as-a-service CRUD endpoints in place of an API you wrote yourself.
 - MongoDB or any non-relational store.
-

What ReturnDesk Must Do

Raising a request

An agent can raise a return request against an order. A request records who the customer is and how to reach them, which order it relates to, which item and how many units, and why it is being returned. The reason is one of: Damaged, Wrong Item, Size Issue, Not As Described, Changed Mind.

Every request carries a unique, human-readable reference that the system generates itself. The person raising the request never types it in.

Finding a request

An agent handling a busy desk needs to find one request among many. They must be able to search by customer, order or reference, narrow the list by status and by reason, sort it, and page through the results. Search, filtering, sorting and paging must all work together, and must all happen on the server. Loading every row into the browser and filtering it there is not acceptable, and we will look at your queries.

Working a request

An agent can open a single request and see everything about it, including every note ever written on it, in order. Notes can be added at any point in a request's life and are never edited or deleted once written.

An agent can move a request through its lifecycle, correct the details of a request that has not yet been decided, and take a request off the desk without destroying the record of it.

The interface

- Status must be immediately readable at a glance in the list.
- Only the actions that are legal for a request in its current state may be offered.
- Search must not fire a request on every keystroke.
- Loading, empty and error states must all be handled. An empty result set should say so rather than render a blank table.
- When the server refuses an action, the user must be told why. Errors must not be swallowed.
- The interface must be usable down to 375px width.

Data to work with

Ship a seed script that fills a clean database with at least 30 requests spread across every status and every reason, with notes on some of them. We will run it, so it must work from scratch.

Business Rules

These rules are the heart of the assignment. A working CRUD app that ignores them will not score well. Every one of them must be enforced by the server. Hiding a button in the interface is not enforcement, and we will test the API directly.

1. Status flow

A request moves through a fixed lifecycle. Only these transitions are legal:

Open → In Review → Approved → Completed
→ Rejected

Rejected and Completed are final. Nothing moves out of them. Any other transition, such as Open straight to Completed, must be refused.

2. Approval needs a resolution

- A request cannot become Approved without a resolution of Refund, Replacement or Store Credit.
- If the resolution is Refund, a refund amount greater than zero is required.
- If the resolution is not Refund, no refund amount may be recorded.

3. One live request per item

A customer cannot have two live requests for the same item on the same order. If a request for that order and item already exists and has not yet reached Rejected or Completed, creating another must be refused. Once the earlier one is closed out, a new request is allowed.

4. Locked once decided

Once a request reaches Approved, Rejected or Completed, its customer and item details can no longer be edited. An attempt must be refused. Notes can still be added at any point.

5. Removal

Taking a request off the desk must never destroy the record. It disappears from the list and can no longer be fetched, but it remains in the database. Only requests that are Open or Rejected may be removed; anything else must be refused.

Refusing properly

When the server refuses something, it must say so honestly and usefully: an appropriate HTTP status code for what actually went wrong, and a consistent, machine-readable body explaining it. Returning 200 with an error buried inside, falling back to 500 for something you could have anticipated, or returning an HTML error page will all count against you.

If You Finish Early

If you complete everything above and still have time, use it as you see fit. What you choose to add, and whether you choose to strengthen what exists rather than add anything at all, tells us something we cannot learn any other way. Note whatever you did in your README.

Rules

1. You have 24 hours from the time this document is emailed to you. The exact deadline is in that email.
2. The work must be your own. You may use documentation, Stack Overflow and AI tools, but you must be able to explain and defend every line in the technical round. We will pick sections of your code and ask you to walk through them, and ask why you chose one approach over another. We will also ask about designs you did not choose.
3. Do not start from a ticketing or CRUD boilerplate repository.

4. Commit as you work. A single commit containing the whole project reads as a copied repository and counts against you.
5. If you cannot finish everything, submit what you have and state clearly in your README what is done, what is not, and what you would do next. An honest partial submission scores well. A README claiming features that do not work scores very badly.
6. No ZIP files. We review from GitHub.
7. If something here is ambiguous, make a reasonable decision and note it in your README. Do not wait for a reply. How you handle the gaps in this brief is part of what we are looking at.

Submission

Reply to the assignment email before the deadline with two links: your deployed application, and a public GitHub repository.

The deployed application

ReturnDesk must be live at a public URL that we can open and use, with its database attached and the seed data loaded. We will click through the working application before we read a single line of your code. Vercel, Render, Railway, Fly and Netlify all have free tiers, as do Neon and Supabase for Postgres.

If you cannot get it deployed in time, submit the repository anyway and explain in your README what went wrong and how far you got. A missing deployment is a real gap, but hiding it is worse.

The repository

A public GitHub repository containing:

- The complete source code.
- The deployed URL, in the README and in the repository description.
- A README.md with setup steps that work from a clean machine, database setup, how to run the seed script, the design decisions you made and why, what is incomplete, and any assumptions you made.
- A .env.example listing every environment variable needed. Never commit your real .env.
- Whatever your project needs to build its database from empty, and the seed script.
- A short note on roughly how many hours you actually spent.

How We Evaluate

Scored out of 100 across these areas.

AREA	WEIGHT	WHAT WE LOOK FOR
Business rules	25	All five rules enforced, and enforced on the server. Illegal transitions, duplicates and locked records all refused correctly and reported clearly.
Data model	20	A schema that fits the problem. Correct types, real constraints, sensible relationships, uniqueness where it belongs. Filtering, sorting and paging done in SQL.
API design	20	A coherent, predictable API. Sensible resources and naming, consistent response and error shapes, honest use of HTTP semantics.

AREA	WEIGHT	WHAT WE LOOK FOR
Frontend	20	Clear and functional interface. Only legal actions offered. All states handled. Server errors surfaced. Responsive.
Code quality	10	Readable structure, sensible naming, validation logic not duplicated across the codebase, no dead code, no secrets committed.
Submission	5	Deployed and working at a public URL. README accurate and complete, setup works from clean, seed data present, commit history shows real progression.

What fails a submission outright

- Data that does not persist in PostgreSQL.
- A repository that cannot be run by following its own README.
- A README describing features that are not implemented.
- Work that is clearly copied wholesale and cannot be explained in the technical round.

Good luck. We would rather see three things built properly than eight built halfway.